

[Sign Up](#)

Two Phase Locking (2PL)

"Concept & Coding" YT Video Notes

Before this video, do checkout the previous "*Distributed Concurrency*" video in this *HLD* playlist....

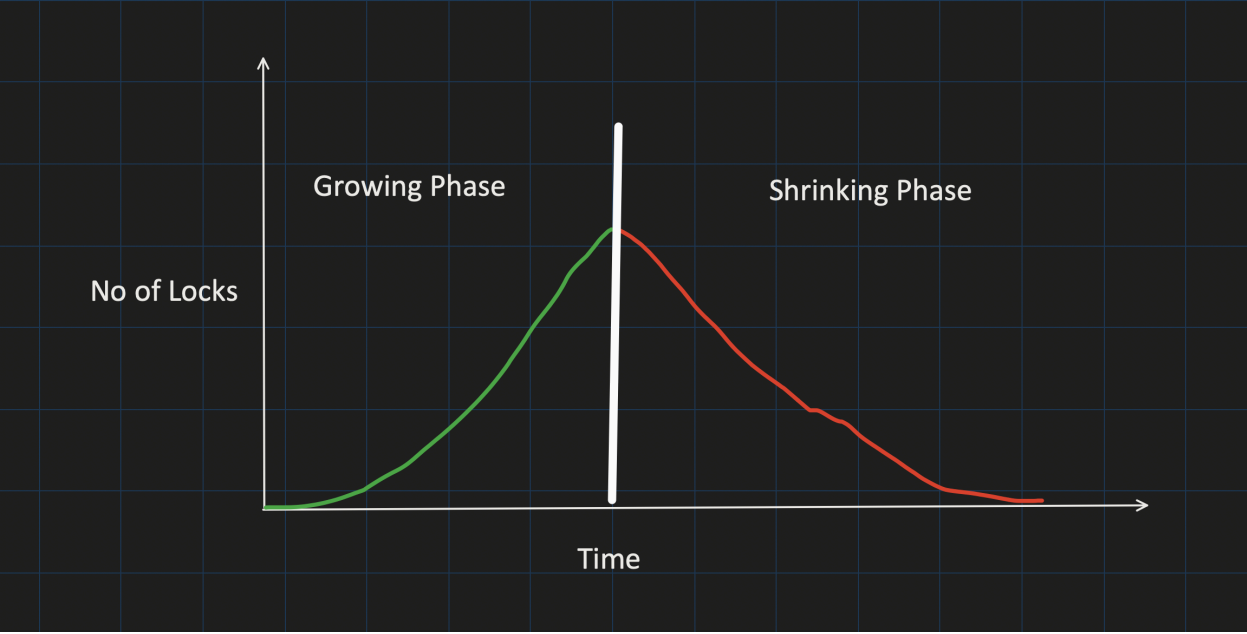
Basic Two Phase Locking:

Phase1: Growing Phase

- Txn request for the lock by Lock manager.
- Lock Manager either grant or denied the lock request (*denied if other txn has placed the locked on it already*)

Phase2: Shrinking Phase

- Txn can not acquire any new locks.
- Txn is only allowed to release the lock which is taken previously.



Time	Transaction 1	Transaction2
t0	BEGIN TXN	
t1	X(A)	
t2	X(B)	
t3	process some work	
t4	Unlock (A)	BEGIN TXN
t5	Unlock (B)	X(B)
t6	COMMIT	X(A)
t7		COMMIT

► (commit will unlock A and B internally)

2 big issue of Basic 2PL and its way around:

1. Deadlock:

Time	Transaction 1	Transaction2
t0	BEGIN TXN	BEGIN TXN
t1	X(A)	X(B)
t2	X(B) (waiting for lock to be released on B)	X(A) (waiting for lock to be released on A)
t3		

OR

Time	Transaction 1	Transaction2
t0	BEGIN TXN	BEGIN TXN
t1	S(A)	S(A)
t2	X(A) (lock conversion only happens if Txn2 Read lock is released)	X(A) (lock conversion only happens if Txn1 Read lock is released)
t3		

Deadlock Prevention Strategies:

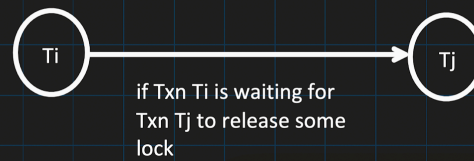
1. Timeout:

In this strategy, scheduler find out that TXN is waiting too long for the lock, it simply assumes that there might be a deadlock involving this transaction and thus abort it.

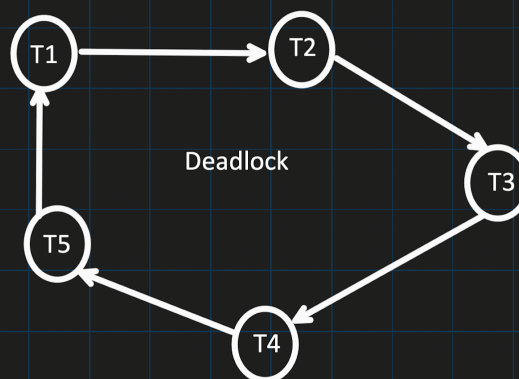
Schedule can make mistake (what if TXN1 wait for the lock, which is acquired by other TXN2 which is taking just long time to finish).

2. Direct graph called Wait-for-Graph(WFG):

There will be an edge from node T_i to T_j .



Scheduler delete the edge, whenever lock is release by particular txn which causing some other txn to wait.



Scheduler looks for cycle in the WFG and try to identify the deadlock.

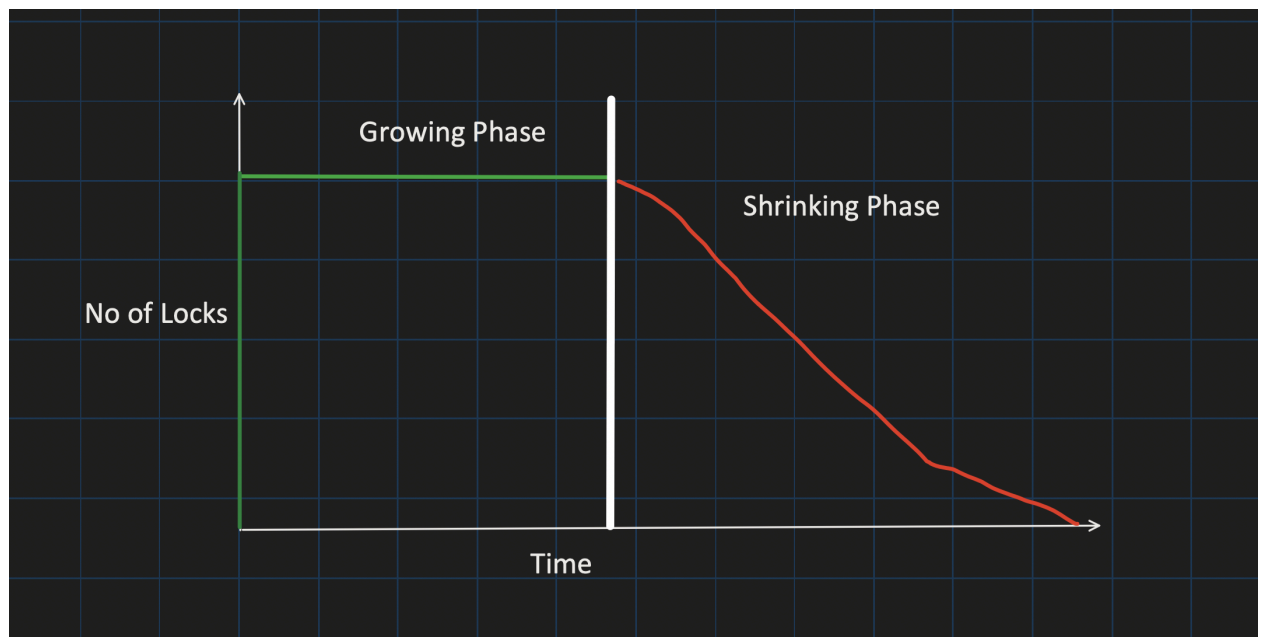
When deadlock is identified, TXN is chosen from the cycle in WFG that need to be ABORTED (these txns are called victim)

Scheduler check below things to identify the victim:

- The amount of effort txn has put till now
- The amount of effort required to finish the txn
- The cost of Aborting the txn (cost generally means here is how many updates already has been done)
- The number of cycles that contains the transaction.

3. Conservative 2PL (also know as Static 2PL):

- Avoid deadlock by requiring each txn to acquire all the locks at start of the txn itself.
- Each txn, predeclared its Read and Write operation to the scheduler.
- Scheduler tries to set all the lock needed by txn.
- If Scheduler fails to acquire any lock, none of the lock will be granted to Txn and it will wait.



Cons of Conservative 2PL:

- Less concurrency
- Extra overhead for Scheduler to know all the Read and Write operation of txn before starting the operation.

4. TimeStamp based Deadlock detection:

Old TimeStamp of Txn = High Priority of the Txn

► Wait-Die :

- Old txn waits for the New txn
- $ts(T1) < ts(T2)$, then T1 will wait for T2 to get completed, otherwise txn will get aborted.

Wound-Wait:

- Old txn give wound to the new txn and make them aborted
- $ts(T1) < ts(T2)$, then T2 abort and releases its lock otherwise txn will wait.

T1	T2
Begin	
	Begin
	X(A)
X(A)	

$ts(T1) < ts(T2)$ = T1 has higher priority than T2

Wait-Die = T1 will wait

Wound-Wait = T2 will be aborted

T1	T2
Begin	
X(A)	
	Begin
	X(A)

$ts(T1) < ts(T2)$ = T1 has higher priority than T2

- Wait-Die = T2 will abort

Wound-Wait = T2 will wait

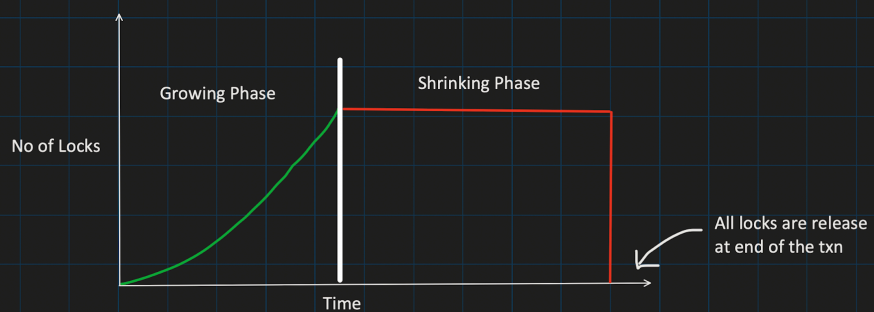
2. Cascading Aborts:

Time	Transaction 1	Transaction2
t0	BEGIN TXN	BEGIN TXN
t1	X(A)	
t2	X(B)	
t3	Update the value of A	
t4	Unlock(A)	
t5		X(A)
t6		Some operation
t7	ABORT	
t8		
t9		This has to be ABORTED Too, as it read the dirty value of A

Cascading Aborts Prevention Strategy:

1. Strong Strict 2PL (also known as Rigorous 2PL):

- Txn is not allowed to acquire or upgrade more lock after growing phase.
- Release all the locks at the end of the txn only.



Cons of Strict 2PL:

- Less concurrency
- Deadlock

Example:

T1: Send 10Rs from A to B

T2: Sum of A balance + B balance

Initial DB values =

A = 100, B = 100

Basic 2PL: (deadlock and cascading abort, both issue present)

Time	Transaction 1	Transaction2
t0	BEGIN TXN	BEGIN TXN
t1	X(A)	
t2	A = A-10 i.e (100-10)	
t3	Update A value	
t4	Unlock(A)	
t5		S(A)
t6		Read value of A i.e 90
t7		Unlock(A)
t8		S(B)
t9		Read value of B i.e 100
t10		Unlock(B)
t11	X(B)	COMMIT
t12	B = B + 10	
t13	Update B value	
t14	Unlock(B)	
t15	Comit	

Output of T2:
A + B = 90+100 = 190

Conservative 2PL: (cascading abort issue present)

Initial DB values =
A = 100, B = 100

Time	Transaction 1	Transaction2
t0	BEGIN TXN	BEGIN TXN
t1	X(A)	
t2	X(B)	
t3	A = A-10 i.e (100-10)	
t4	B = B+10 i.e (100+10)	
t5	Update B value	
t6	Unlock (A)	
t7		S(A)
t8		S(B) (waiting.....)
t9		(waiting.....)
t10	Unlock (B)	S(B) (Success)
t11	Commit	Read value of A and B
t12		Unlock (A)
t13		Unlock (B)
t14		Commit
t15		

Output of T2:
 $A + B = 90 + 110 = 200$

Strong Strict 2PL:

Initial DB values =
A = 100, B = 100

Time	Transaction 1	Transaction2
t0	BEGIN TXN	BEGIN TXN
t1	X(A)	
t2	A = A-10 i.e (100-10)	S(A) (waiting...)
t3	Update A value	
t4	X(B)	
	b = B+10 i.e (100+10)	
t5	Update B value	
t6	Commit	
t7		S(A) (Success)
t8		Read A value
t9		S(B)
t10		Read B Value
t11		Commit
t12		
t13		
t14		
t15		

Output of T2:
 $A + B = 90 + 110 = 200$

Report Abuse